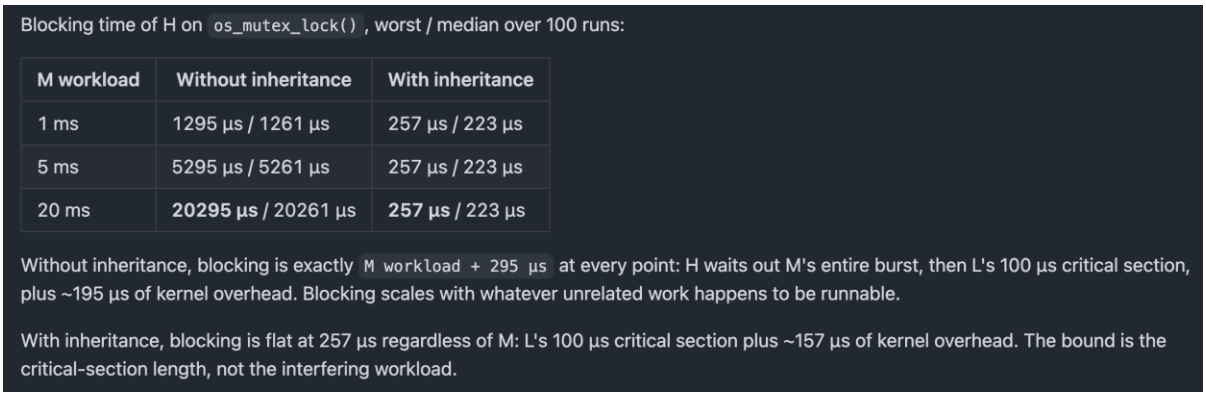
1.



Blocking time of H on `os_mutex_lock()`, worst / median over 100 runs:

M workload	Without inheritance	With inheritance
1 ms	1295 µs / 1261 µs	257 µs / 223 µs
5 ms	5295 µs / 5261 µs	257 µs / 223 µs
20 ms	**20295 µs** / 20261 µs	**257 µs** / 223 µs

Without inheritance, blocking is exactly `M workload + 295 µs` at every point: H waits out M's entire burst, then L's 100 µs critical section, plus ~195 µs of kernel overhead. Blocking scales with whatever unrelated work happens to be runnable.

With inheritance, blocking is flat at 257 µs regardless of M: L's 100 µs critical section plus ~157 µs of kernel overhead. The bound is the critical-section length, not the interfering workload.

KoRTOS is a preemptive priority-based RTOS kernel I wrote from scratch for Cortex-M4, no CMSIS and no vendor HAL, down to the PendSV context switch in inline assembly. The part I'm actually proud of is the table. I wrote a sample app that deliberately induces priority inversion and measured worst-case blocking using DWT. Without inheritance the blocking grows with whatever the interfering task is doing, 20.3 ms for a 20 ms interferer. With inheritance it sits at 257 µs no matter what the interferer does, because all that's left is the critical section plus kernel overhead.

2.

When I first got started in embedded I did everything in STM32CubeIDE. I was a naive child and didn't know about the wonderful world of editing in VSCode. It was during my last co-op that I started writing all the code in VSCode, since I was in a bigger codebase and VSCode makes navigation so much easier. CubeIDE is Eclipse from about a decade ago and writing code in it is a pain, and the thing that finally switched me over is that searching across files is awful. So CubeIDE became just a flashing and debugging front end for me. That was an alright split during my co-op, since ZeroPilot uses the HAL and ioc generation anyway, so the breakpoints and Live Expressions UI fit fine. It's weirder with KoRTOS, which avoids CMSIS and the vendor HAL, doesn't use ioc generation, and is register level throughout. I'm sitting in CubeIDE while rejecting basically everything it wants to generate for me. Scrolling Instagram reels is how I found out there's an STM32 VSCode extension (I did actually learn something from reels), so I've been trying to move fully into VSCode since ST's extension does live watch and the register tree now too.

3.

I haven't debugged a random reset specifically, but I've spent enough time on intermittent hardware bugs to know how I'd start.

The first thing I'd want to know is whether the chip actually lost power, or whether power was fine and something restarted it on purpose. Those lead to completely different investigations. Power loss means I'm looking at hardware: the supply, decoupling, current spikes, connectors. If power was fine then something restarted it deliberately, which usually means the firmware hung and the watchdog fired, or it crashed.

On STM32 there's a register that latches the reset reason, with separate bits for watchdog, software reset, the NRST pin, power-on, and brownout. Reading that is probably the cheapest thing you can do and it splits the search immediately.

Then I'd want some kind of logging. On a board that resets every couple of hours, it's not worth just sitting there waiting for it to happen. At minimum I'd log uptime, so I can see whether the failures land after a consistent period or whether they're random. If it's consistent, that points at something periodic, like a counter overflowing. If it's scattered, it's more likely a race condition that only fails under the right combination of conditions. And if there are spare boards, it's worth running several at once so I'm not waiting three hours per attempt.

I'd change strategy when the measurements stop telling me anything, like running three boards for a day and getting completely random results. At that point I'd stop waiting for the failure and try to force it instead.

The closest real incident I have isn't a reset, but it's the one that taught me the most about bugs that live between the code and the board. During my last co-op our IMU driver worked fine on the dev board, then returned garbage on our custom PCB. The two boards used different MCUs, so the pin configuration and peripheral setup had to change, but the IMU was the same part and the driver logic was unchanged and already proven. So my instinct was that it was the board or the thin layer of config around the driver, not the driver itself.

The way to check that is to look at the bus directly and compare what's physically on the wire against what the driver thinks it's reading. There were no test points, so every line I wanted to see meant scraping soldermask off a via. That's annoying enough that I only did three, CLK, MISO and MOSI, because that's where the data is. It felt like the obvious choice, and I didn't think about it much harder than that. That decision cost me the next two days.

That did tell me something. The garbage on the wire matched what the debugger showed the driver receiving, so the driver was faithfully reading what it was given and the problem was on the bus, not in the parsing. What it couldn't tell me was why the bus was wrong,

since garbage on MISO is equally consistent with contention, a wrong SPI mode, or a dead device. At that point I decided the hardware was probably fine and went back to debugging the software.

What cracked it was an accident. I set a breakpoint during init, sat on it for a bit, and when I continued the driver just worked. Adding a half second delay in init made it work every time. That made no sense. Nothing in a SPI driver needs half a second to settle, and a fix that arbitrary means you don't understand the mechanism yet.

Chasing why half a second mattered is what led to the actual cause. The custom board had two IMUs sharing the bus where the dev board had one jumper-wired in externally, and we were only driving the CS pin for the device we were talking to. The other CS was left floating, so nothing was driving it and its voltage was set by leakage current charging the trace capacitance. That's nanoamps into tens of picofarads, which is why it took hundreds of milliseconds to drift past the second IMU's logic-high threshold. Until it got there that device still thought it was selected and was driving MISO alongside the one we wanted. The delay worked because it was waiting for a floating line to leak its way up to a valid high.

The bug was in software, but no amount of reading the driver would have found it. The code was doing exactly what it said, and the problem was in what it never said anything about. Every pin on the board is in some state whether the firmware sets it or not, and the ones nobody wrote a line about are exactly the ones you won't find by reading code.

4.

"Don't reinvent the wheel, just use the vendor HAL." I think that's good advice about shipping and bad advice about learning, and people hand it out like it covers both.

I wrote an RTOS kernel from scratch on a Cortex-M4 with no CMSIS or vendor HAL, down to the linker script and startup code. Not because I thought I'd beat FreeRTOS. It's that once you've written a context switch and a priority inheritance mutex yourself, that layer stops being somewhere things mysteriously go wrong. The IMU bug is the same idea one level down, the driver was fine and the answer was in what the config never touched.

So what I'd actually defend is narrower than "write everything yourself." Use the HAL, ship the thing, but read it once and know what it's doing to the registers underneath. Knowing what's below the API you're calling is what turns a weird bug from guessing into debugging, and you don't get that from being told not to reinvent things.

5.

Not a hack exactly, more a fix I shipped knowing it was covering something else. ZeroPilot's self-leveling mode was doing uncommanded jerks. I tuned it in SITL first, then figured the gains might not translate to hardware, so we got a tuning rig and tuned it again there. Still did it. Then I went through the gyro path, adding hardware AAF, low pass, bias calibration, and an FFT harmonic notch for motor vibration. That made it fly noticeably better, completely stable for a few seconds at a time, but it would still throw a random jerk in between. What was actually happening is that the Mahony filter was weighting the accelerometer too heavily, so high frequency noise and vibration in the accel path were corrupting the attitude estimate.

The fix was turning those gains down until it rejected the spikes. That worked and it flew great after. I was aware that theoretically the tradeoff was slower drift correction but didn't worry too much about it since it was flying so well. However, it showed up exactly as theory said it would, in SITL after it shipped, where I could fly much longer than on a battery, and I could see it stop returning to perfectly level after a while. On a real flight, you never fly long enough to notice. The issue also reemerged when we ported to an AHRS EKF, which told me the data underneath was still bad and the Mahony gains had just been hiding it.